

ADR - Choix de MongoDB pour les événements

1. Contexte

L'application Pigeon enregistre plusieurs types d'événements : les connexions, les paiements, les erreurs applicatives, les appels à son API et les notifications. Ces événements sont différents et ne contiennent pas les mêmes informations. Par exemple, un paiement possède un montant et un statut, alors qu'un appel API possède un endpoint et un temps de réponse.

2. SQL

Avec SQL, on pourrait créer une table pour chaque type d'événement.

Avantages :

- Les données sont bien organisées.
- Les relations entre les données sont faciles à gérer.
- Les contraintes permettent de garder des données cohérentes.
- SQL est adapté lorsque les données ont une structure stable.

Inconvénients :

- Les différents types d'événements ont des structures différentes.
- Il faudrait créer plusieurs tables.
- L'ajout d'un nouveau type d'événement peut demander de modifier la base.
- Le modèle est donc moins flexible pour notre projet.

3. NoSQL avec MongoDB

Avec MongoDB, les événements peuvent être stockés dans une collection sous forme de documents. Les documents peuvent avoir des informations différentes tout en gardant des champs communs comme eventType, timestamp et userId.

Avantages :

- La structure est flexible.
- Plusieurs types d'événements peuvent être stockés dans la même collection.
- Il est plus facile d'ajouter de nouvelles informations.
- MongoDB permet d'utiliser des pipelines d'agrégation pour analyser les événements.

Inconvénients :

- Les relations entre les données sont moins adaptées qu'en SQL.
- Il y a moins de contraintes pour contrôler la structure des données.
- Il faut donc faire attention à garder des documents cohérents.

4. Comparaison

SQL est intéressant lorsque les données sont bien structurées et qu'il y a beaucoup de relations entre elles. MongoDB est plus flexible lorsque les données peuvent avoir des structures différentes. Dans notre projet, les événements de Pigeon sont justement de types différents et ne contiennent pas tous les mêmes informations. MongoDB correspond donc mieux à ce besoin.

5. Décision

Nous choisissons MongoDB principalement pour sa flexibilité. Elle permet de stocker différents types d'événements dans une même collection tout en conservant des champs communs pour pouvoir les analyser.